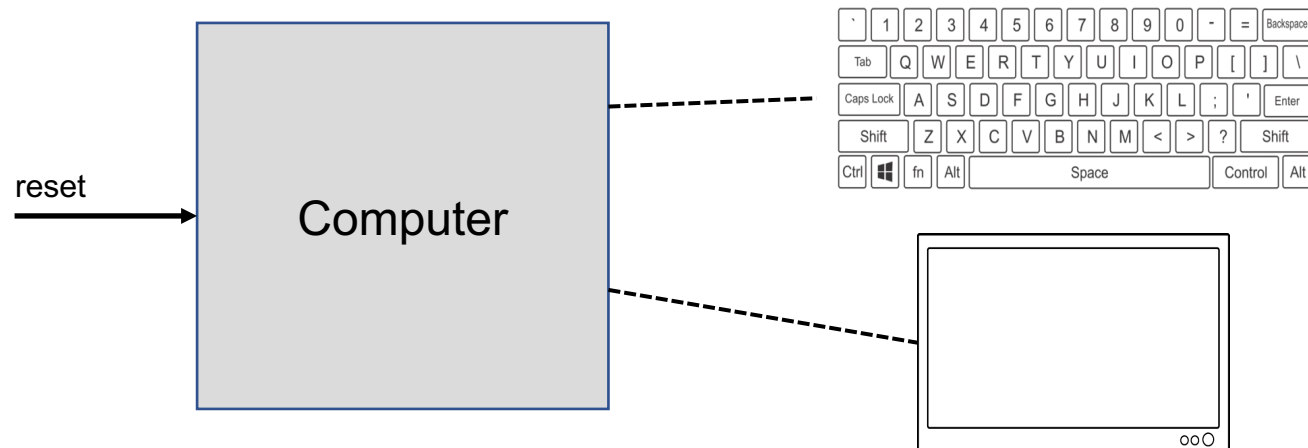


Realizzazione del Computer Hack



A che punto siamo

Completato

Circuiti logici

- Nand
- Not
- And / Or
- Xor
- Mux
- Dmux
- Not16
- And16
- Or16
- Mux16
- Or8Way
- Mux4Way16
- Mux8Way16
- DMux4Way
- DMux8Way

Completato

Circuiti aritmetici

- HalfAdder
- FullAdder
- Add16
- Inc16
- ALU

Da fare

Realizzazione del Computer

- Memory
- CPU
- Computer

Completato

Circuiti sequenziali

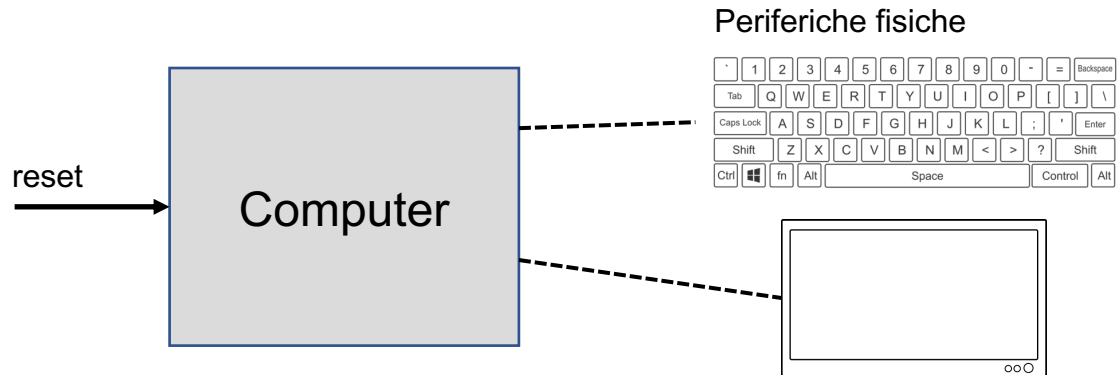
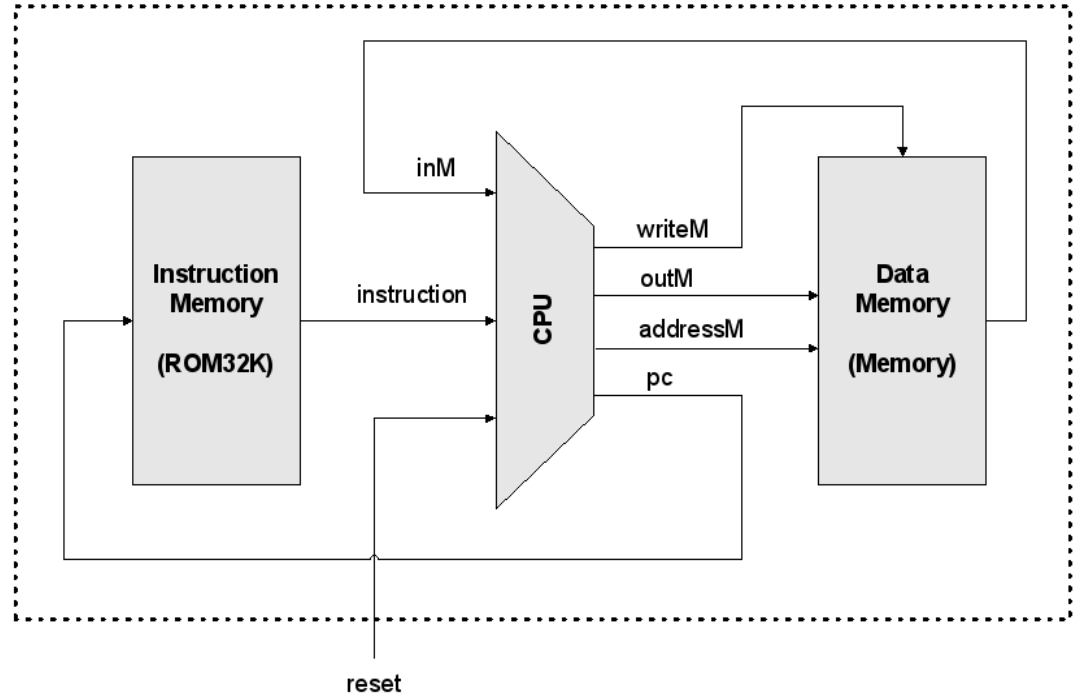
- DFF
- Bit
- Register
- RAM8
- RAM64
- RAM512
- RAM4K
- RAM16K
- PC

Il computer Hack

- Una architettura di tipo Von Neumann a 16-bit
- Con separazione fra memoria dati e memoria programma (per poter caricare contemporaneamente dati e istruzioni – questo dettaglio è da architettura di tipo “Harvard” più che di Von Neumann). Se così non fosse (come in computer moderni), servirebbero almeno due cicli di clock per ogni istruzione (perché il bus dati verso la memoria avrebbe due utilizzi distinti, per forza non contemporanei).
- Schermo bianco e nero composto da 256 righe e 512 colonne
- Tastiera
- Progettato per eseguire programmi scritti nel linguaggio macchina Assembly Hack (binario)
- Di facile realizzazione a partire dall'insieme dei circuiti logici già costruiti e studiati fino ad ora

Componenti principali del computer Hack

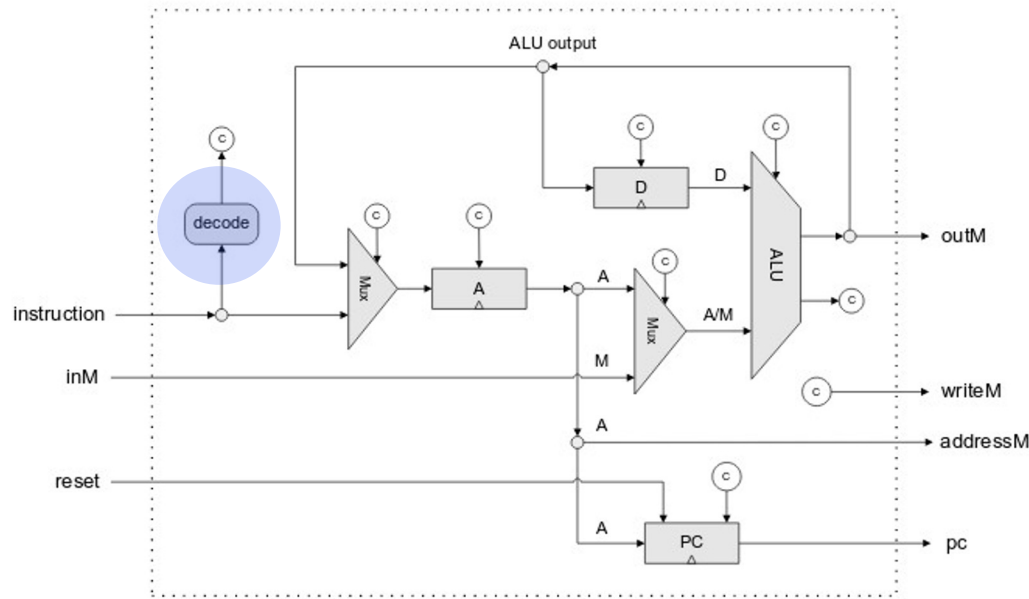
- Memoria istruzioni (ROM)
- Memoria dati (Memory):
 - Dati (RAM)
 - Schermo
 - Tastiera
- CPU
- Computer completo



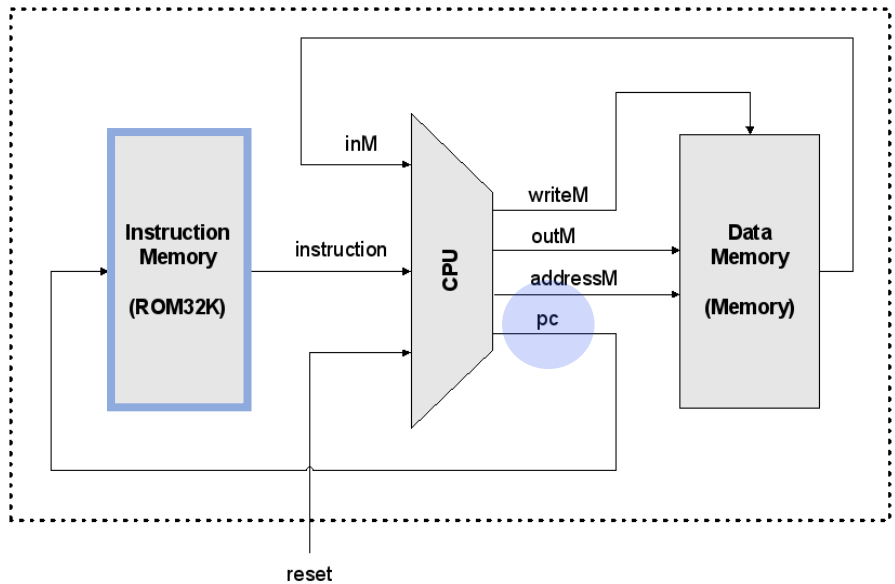
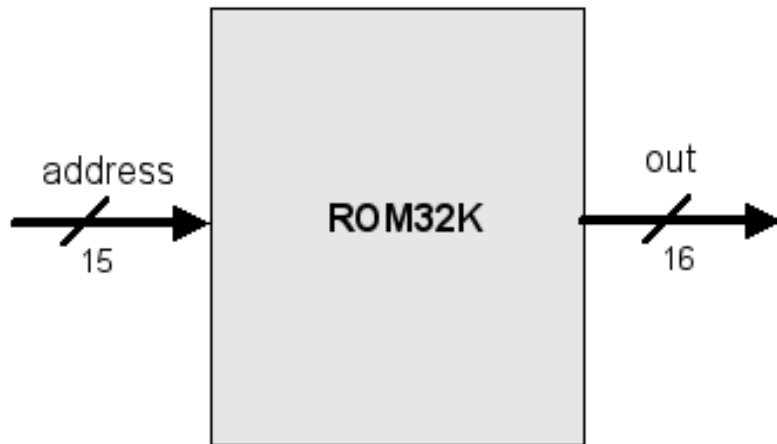
Il computer Hack

Lo scopo di queste slide è illustrare i chip del computer Hack non ancora studiati, e chiarire alcuni dettagli relativi a quelli già visti finora, allo scopo di facilitare la realizzazione progettuale dell'intero computer (il prossimo progetto).

Aggiungiamo tuttavia dettagli sulla codifica in binario delle istruzioni Assembly Hack (simboliche) operata dall'assemblatore, perché questo ci serve per capire come realizzare il circuito responsabile per la fase Decode (e quindi anche le fasi Execute e Fetch), che finora abbiamo sempre rappresentato in maniera informale nei diagrammi (vedi figura). In seguito vedremo esattamente come funziona un assemblatore.



Memoria per le istruzioni (ROM)



- La ROM (un chip *built-in*) è precaricata con un programma scritto nel linguaggio Assembly Hack
- Dato un indirizzo ROM address a **15-bit** (proveniente dal PC), emette un numero a 16-bit che codifica in binario l'istruzione corrente da eseguire, ovvero $\text{out} = \text{ROM32K}[\text{address}]$

Chip Memory

Una precisazione: finora, per semplicità, ci siamo sempre riferiti a questo componente come memoria RAM (ed infatti, anche programmando in Assembly, ci siamo sempre riferiti ad un indirizzo di memoria k come $\text{RAM}[k]$).

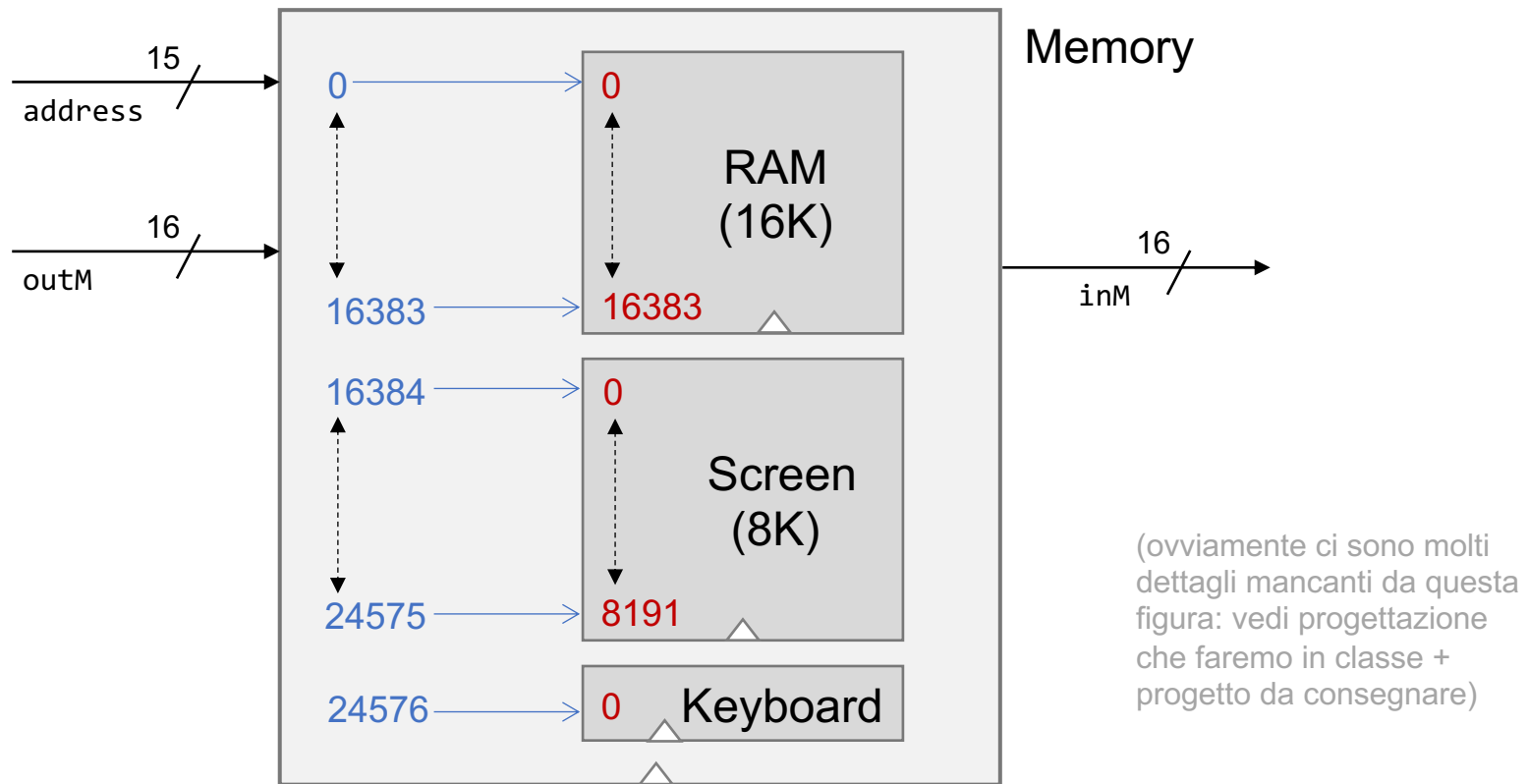
In realtà, come vedremo ora, il chip Memory è composto da 3 componenti per una dimensione totale di 24K:

- Il chip **RAM** (16K): la memoria RAM di lavoro vera e propria
- Il chip **Screen** (8K): memoria tipo RAM usata per la *mappa dello schermo*
- Un registro **Keyboard**: un singolo registro per leggere il tasto premuto

Per questo motivo, quando necessario, possiamo riferirci al registro Memory con indirizzo k come **MEM[k]**, senza specificare se si tratti di memoria dati RAM, di indirizzo relativo allo schermo, o alla tastiera.

Chip Memory

In blu: lo spazio degli indirizzi del chip Memory (univoco: da 0 a 24576).
In rosso: gli spazi di indirizzamento di ciascun chip interno.



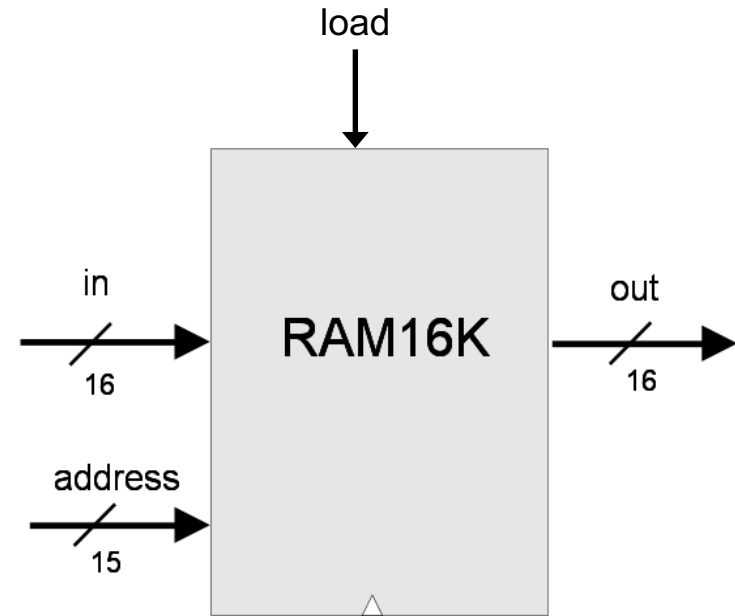
Ricordare le due variabili predefinite in Hack: SCREEN=16384, KBD=24576 (vedi slide ISA).

Domanda: un indirizzo a 15 bit può indirizzare fino a $2^{15}-1=32767$. Cosa accade agli indirizzi mancanti dalla figura (esprimibili in binario, ma non corrispondenti a chip fisici)?

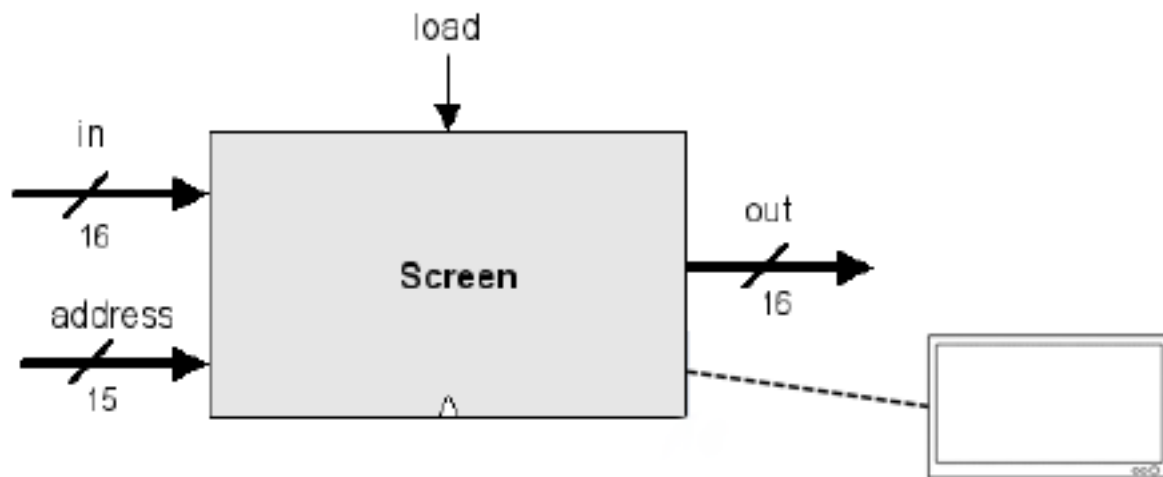
Memoria per i dati (RAM)

Logica di lettura/scrittura:

- Per **leggere** $\text{RAM}[k]$:
 - portare k sul bus indirizzi
 - leggere out
- Per **scrivere** $\text{RAM}[k] = \text{value}$:
 - portare k sul bus address
 - portare value sul bus in
 - portare 1 su load
 - attendere un ciclo di clock



Schermo



Lo schermo ha una funzionalità tipo RAM:

- Logica di lettura:
$$\text{out} = \text{Screen}[\text{address}]$$
- Logica di scrittura:
$$\text{if load then Screen}[\text{address}] = \text{in}$$

Esiste una corrispondenza diretta fra ogni pixel del monitor ed i corrispondenti bit memorizzati ad opportuni indirizzi all'interno del chip Screen.

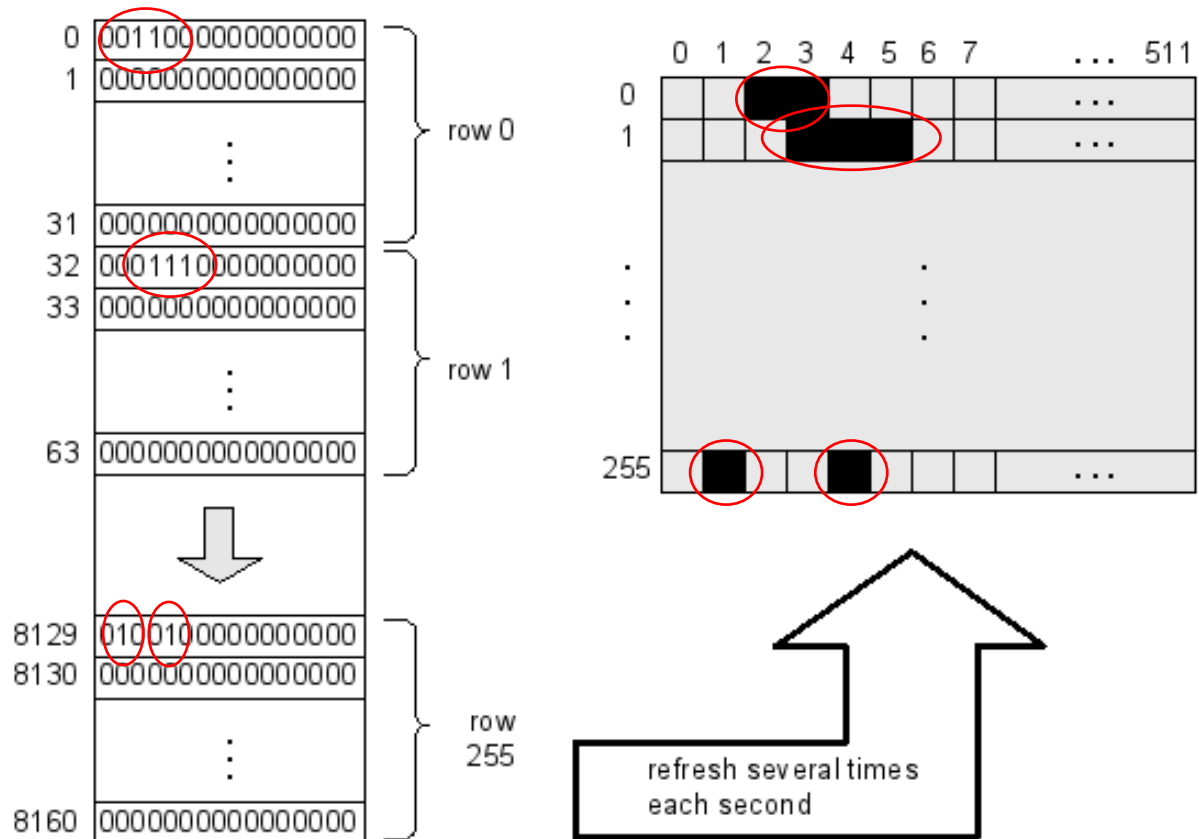
Periferica schermo **simulata**:

The screenshot shows a hardware simulator window with a "Screen" component selected. An orange arrow points from a text box to the simulated screen area. The text box contains the text "Schermo B&W 256 per 512".

Quando il chip built-in Screen.hd1 viene caricato nell'hardware simulator, viene aperta una finestra che simula lo schermo; viene frequentemente "refreshed" per rispettare la relativa mappa in memoria

“Mappa dello Schermo” sulla memoria “Screen”

Il chip Screen contiene
8K parole da 16 bit.
Un bit a 1 indica che il
corrispondente pixel
deve essere nero,
0 indica invece bianco.



Per impostare il pixel (row, col) dello schermo per essere nero o bianco:

Settare il bit $col\%16$ della parola alla locazione $Screen[row*32+col/16]$, dove $col/16$ è una divisione intera, a 1 (nero) oppure 0 (bianco)

Esempio di uso dello schermo in linguaggio Hack

Disegna sullo schermo un rettangolo largo 16 pixel e alto MEM[0] pixel.

Esempio di uso dello schermo in linguaggio Hack

Disegna sullo schermo un rettangolo largo 16 pixel e alto MEM[0] pixel

```
@0
D=M           // D=MEM[0]
@INFINITE_LOOP
D;JLE         // if D>0 then
@counter
M=D           // counter=D
@SCREEN      // predefinita
D=A
@address
M=D           // address=SCREEN
(LOOP)
@address
A=M           // A=address
M=-1          // MEM[A]=111...1
@32
D=A
@address
M=M+D         // address+=32
```

```
@counter
MD=M-1       // counter--, scrivo anche D per test
@LOOP
D;JGT         // ciclo se counter<=0
(END)
@END
0;JMP
```

vedi file rect.asm

Tastiera



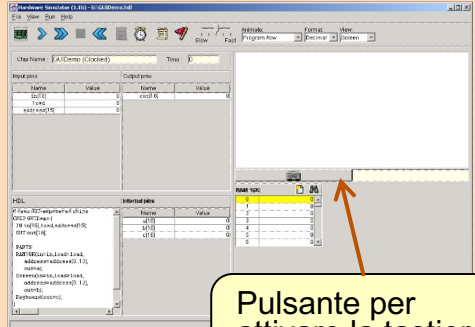
Chip Keyboard: un unico registro da 16-bit

Out: la codifica ASCII (estesa a 16-bit) del tasto correntemente premuto, oppure 0 se non si preme alcun tasto (esistono alcuni tasti speciali, vedi sotto)

Tasti speciali:

Key pressed	Keyboard output	Key pressed	Keyboard output
newline	128	end	135
backspace	129	page up	136
left arrow	130	page down	137
up arrow	131	insert	138
right arrow	132	delete	139
down arrow	133	esc	140
home	134	f1-f12	141-152

Tastiera **simulata**:



Pulsante per attivare la tastiera simulata

Esiste un chip built-in Keyboard.hd1. Quando viene caricato nel simulatore, si collega alla tastiera fisica e collega il codice del tasto correntemente premuto alla mappa in memoria implementata dal chip Keyboard

Per leggere dalla tastiera: leggere il contenuto del chip Keyboard.

Nota: il registro non accetta valori in scrittura: possiamo solo leggere.

Esempio di uso della tastiera in linguaggio Hack

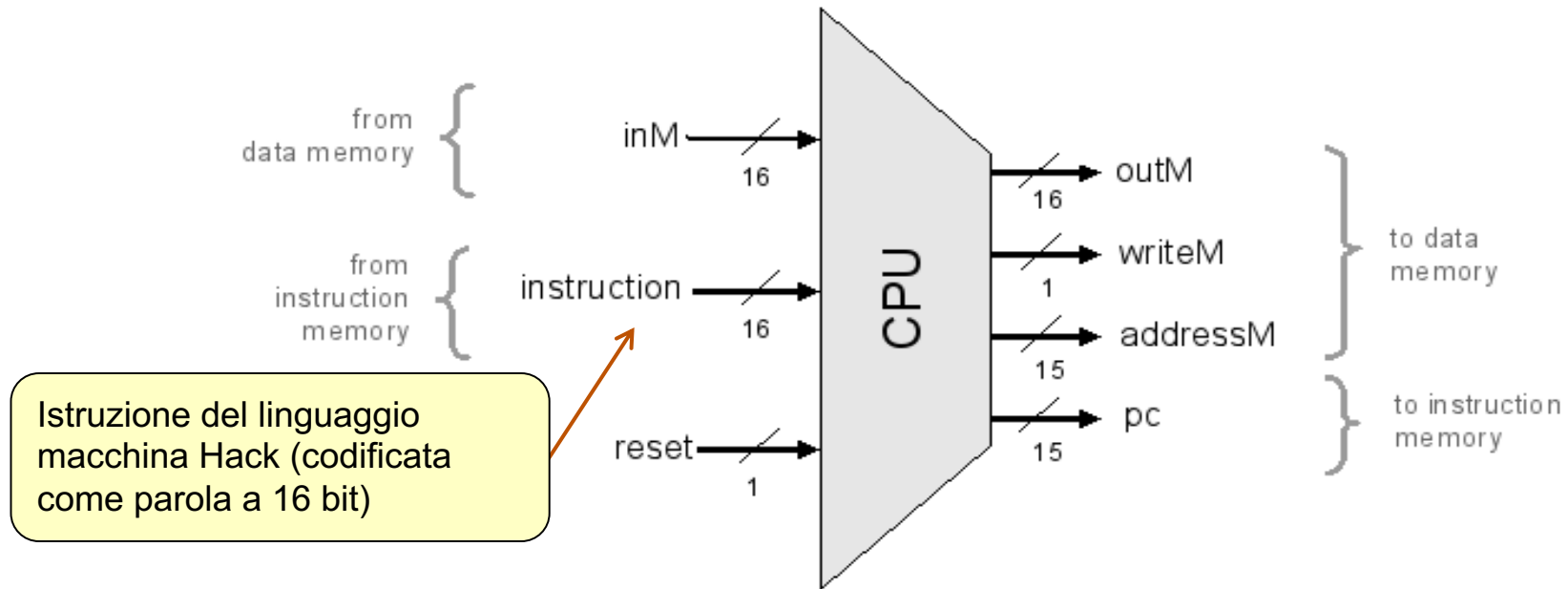
Colora progressivamente lo schermo di nero se si tiene premuto un tasto. Logica: iteriamo (LOOP) su tutte le locazioni della mappa dello schermo, dalla prima all'ultima; se durante l'iterazione un tasto è premuto allora scriviamo -1 (16 '1') all'indirizzo corrente, altrimenti 0 (16 '0'). Quando il ciclo raggiunge l'ultima locazione della mappa (**) ricominciamo il programma puntando di nuovo alla prima locazione. Codice in `fill.asm`

```
(START)
  @SCREEN                // 16348
  D=A
  @i
  M=D    // i punta a 1a word della mappa
(LLOOP)                // while(true)
  @i
  D=M
  @24575    // ultima word della mappa
  D=A-D      // D=24575-i
  @START
  D;JLT      // (**) se D<0 ricomincia
  @KBD
  D=M
  @COLORANERO
  D;JNE    // salta se un tasto è premuto
```

```
  @i
  D=M
  A=D
  M=0      // MEM[i]=00...0 (bianco)
  @CONTINUE
  0;JMP
(COLORANERO)
  @i
  D=M
  A=D
  M=-1     // MEM[i]=11...1 (nero)
(CONTINUE)
  @i
  M=M+1    // i++: prossimo indirizzo
  @LOOP
  0;JMP    // fine while
```

Esercizio: eseguire un ciclo infinito per controllare se un tasto è premuto; in questo caso, uscire dal ciclo ed avviare un ciclo di colorazione di tutto lo schermo che non può essere interrotto.

Comportamento della CPU

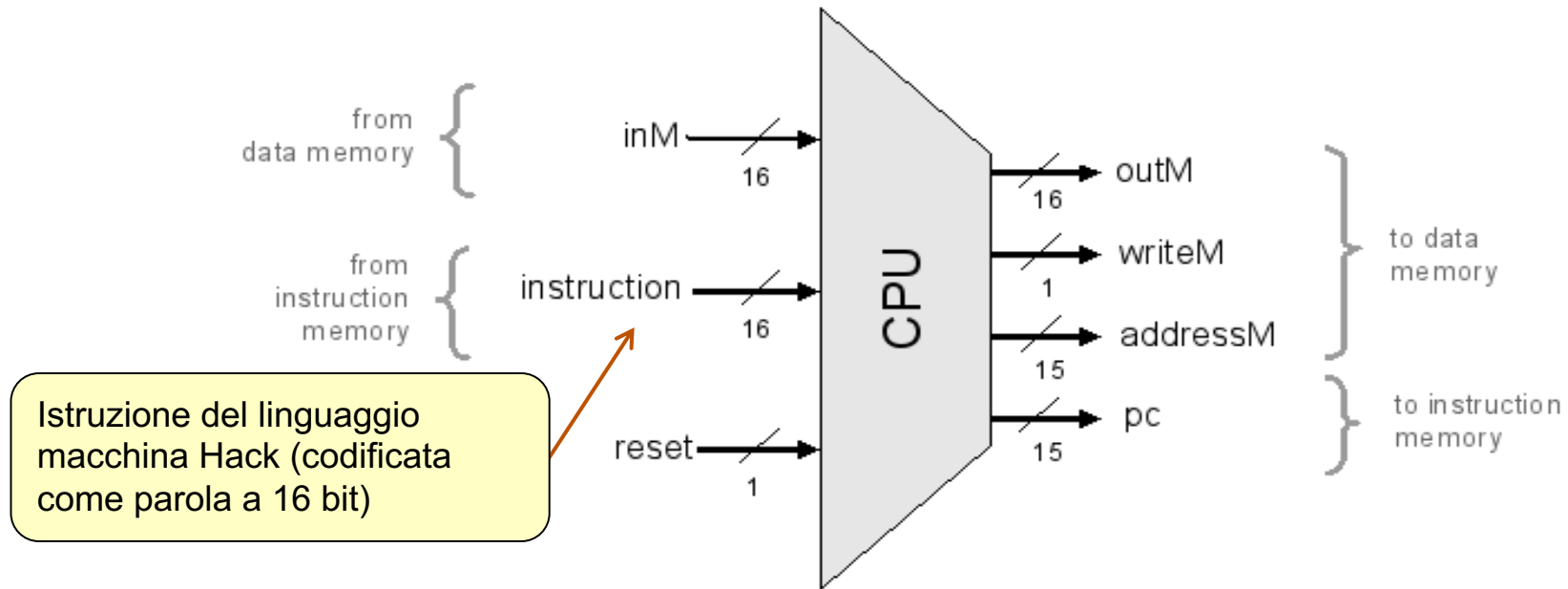


Componenti interni della CPU: ALU e 3 registri: A, D, PC

La CPU esegue la A/C-instruction seguendo la specifica del linguaggio Hack:

- I valori D e A, se presenti nell'istruzione, sono letti/scritti da/in **A**, **M**
- Il valore M, se presente nella parte destra dell'istruzione, viene letto da **inM**
- Se la parte sinistra dell'istruzione contiene M, l'output della ALU viene posto su **outM**, il valore di A viene posto su **addressM**, e **writeM** viene settato ad 1

Comportamento della CPU



Politiche di caricamento delle istruzioni:

Se si tratta di una C-instruction (dest=comp;jump), instruction può includere un salto; nel caso di salto condizionato la condizione dipende dai due bit di controllo della ALU (zr e ng), indicanti se l'output della ALU è zero o negativo.

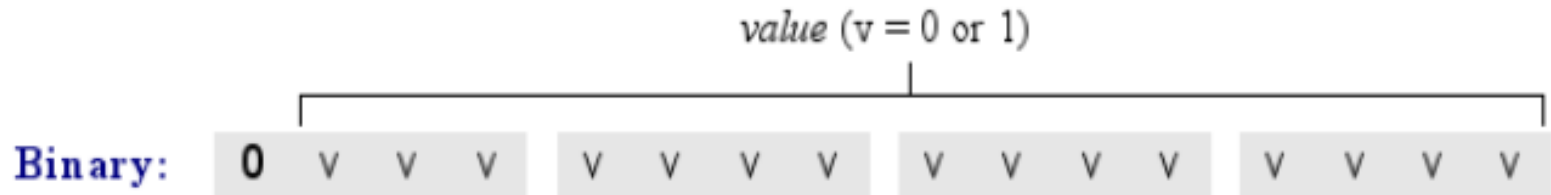
Quindi:

Se **reset==0**: la CPU controlla se va fatto il jump. In caso di jump si carica il contenuto di A in PC ($PC=A$); altrimenti, si incrementa il registro ($PC=PC+1$)

Se **reset==1**: si mette 0 in PC (si fa ripartire il computer)

Codifica delle A-instruction

Symbolic: `@value` *// Where value is either a non-negative decimal number*
 // or a symbol referring to such number.



Il primo bit, uguale a 0, indica che si tratta di una A-instruction: quindi il numero “value” viene codificato in binario dai successivi **15** bit.

Questo spiega perché gli indirizzi sono a 15 bit e non 16 bit (vedi le interfacce dei chip nelle slide precedenti).

Range di valori codificabile in *value*: i numeri **non negativi** che si possono codificare in complemento a 2 con 16 bit (i numeri negativi hanno infatti il bit più significativo ad 1).

Codifica delle C-instruction

dest = comp; jump

comp

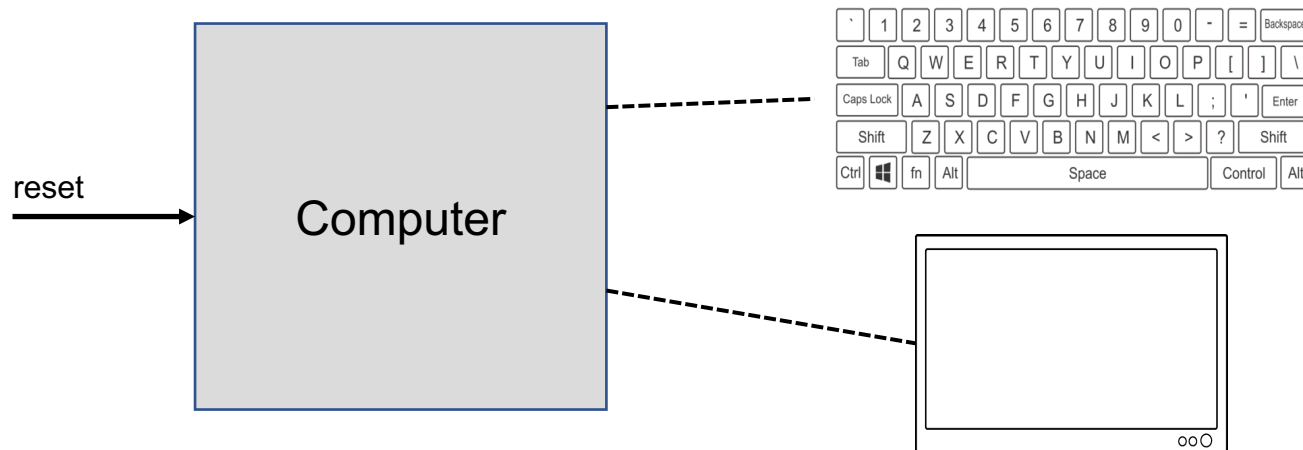
dest

jump

binary: 1 1 1 a c1 c2 c3 c4 c5 c6 d1 d2 d3 j1 j2 j3

(when a=0) <i>comp</i>	c1	c2	c3	c4	c5	c6	(when a=1) <i>comp</i>	d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	1	0	1	0	1	0		0	0	0	null	The value is not stored anywhere
1	1	1	1	1	1	1		0	0	1	M	Memory[A] (memory register addressed by A)
-1	1	1	1	0	1	0		0	1	0	D	D register
D	0	0	1	1	0	0		0	1	1	MD	Memory[A] and D register
A	1	1	0	0	0	0	M	1	0	0	A	A register
!D	0	0	1	1	0	1		1	0	1	AM	A register and Memory[A]
!A	1	1	0	0	0	1	!M	1	1	0	AD	A register and D register
-D	0	0	1	1	1	1		1	1	1	AMD	A register, Memory[A], and D register
-A	1	1	0	0	1	1	-M					
D+1	0	1	1	1	1	1			j1	j2	j3	
A+1	1	1	0	1	1	1	M+1	(out < 0)	(out = 0)	(out > 0)	Mnemonic	Effect
D-1	0	0	1	1	1	0		0	0	0	null	No jump
A-1	1	1	0	0	1	0	M-1	0	0	1	JGT	If out > 0 jump
D+A	0	0	0	0	1	0	D+M	0	1	0	JEQ	If out = 0 jump
D-A	0	1	0	0	1	1	D-M	0	1	1	JGE	If out ≥ 0 jump
A-D	0	0	0	1	1	1	M-D	1	0	0	JLT	If out < 0 jump
D&A	0	0	0	0	0	0	D&M	1	0	1	JNE	If out ≠ 0 jump
D A	0	1	0	1	0	1	D M	1	1	0	JLE	If out ≤ 0 jump
								1	1	1	JMP	Jump

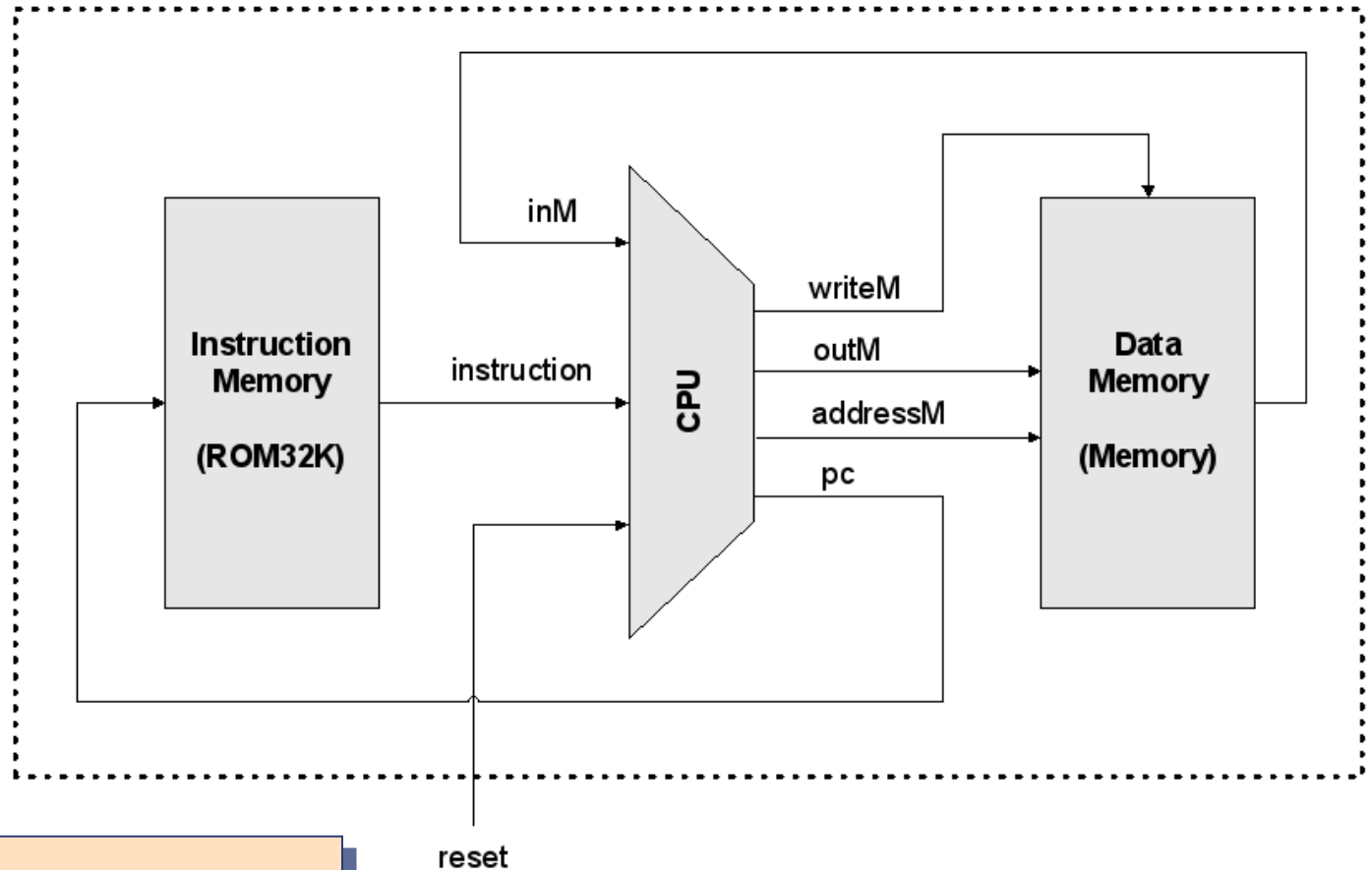
Chip Computer



Chip Name: Computer // Topmost chip in the Hack platform
Input: reset
Function: When reset is 0, the program stored in the computer's ROM executes. When reset is 1, the execution of the program restarts. Thus, to start a program's execution, reset must be pushed "up" (1) and "down" (0).

From this point onward the user is at the mercy of the software. In particular, depending on the program's code, the screen may show some output and the user may be able to interact with the computer via the keyboard.

Chip Computer



```
CHIP Computer {
  IN reset;
  PARTS:
    // TO-DO
}
```